UNIVERSITEIT VAN PRETORIA
UNIVERSITY OF PRETORIA
YUNIBESITHI YA PRETORIA

Engineering, Built Environment and IT
Department of Computer Science
Imperative Programming
COS 132

Examination Opportunity 3

24 June 2023 (11:30 to 14:00)

Each Q – new pg

Examiners

Internal: Mr Pula Rammoko, Mr Sheanesu Makura, Ms Seani Rananga, Prof Stefan Gruner and Dr Linda Marshall
External: Prof Jan Eloff

Instructions

1. Read the question paper carefully.
2. Answer all the questions in the answer book provided. All answers must be written indelible pen.
3. Make sure you clearly mark where each question begins.
4. The assessment opportunity comprises of 8 questions on 8 pages. Refer to the table below for the mark allocation per question.
5. Two (2) hours and 30 minutes have been allocated for you to complete this paper.
6. A simple calculator may be used when writing the paper. No other electronic device may be used or in your possession.
7. In the event of loadshedding, extra time will be given depending on how long it takes the generators to kick in.
8. You may bring your own notes (hand written or typed) into the examination room.
9. Your solutions should work for any valid input for the given question, not just the sample inputs given for the question. Do not "hardcode" the answer.

Integrity statement:
The University of Pretoria commits itself to produce academic work of integrity. I affirm that I am aware of and have read the Rules and Policies of the University, more specifically the Disciplinary Procedure and the Tests and Examinations Rules, which prohibit any unethical, dishonest or improper conduct during tests, assignments, examinations and/or any other forms of assessment. I am aware that no student or any other person may assist or attempt to assist another student, or obtain help, or attempt to obtain help from another student or any other person during tests, assessments, assignments, examinations and/or any other forms of assessment.

Question:	1	2	3	4	5	6	7	8	Total
Marks:	10	10	15	8	8	7	8	11	77

An achievement of 70 marks out of 77 is considered as full marks (100%).

Short questions

Question 1 [10 marks]

For each of the following multiple choice questions, write down the most appropriate option.

(a) Given the following C++ code: (1)

```
#include <iostream>
using namespace std;
int main() {
    int aa=20, bb, cc;
    bb = aa++;  bb = 20  aa = 21
    cc = aa; cc=21
    cout << aa << " ";  21
    cout << bb << " ";  20
    cout << cc;  21
    return 0;
}
```

When the above program is run, what will be the output?

A. 20 20 20
B. 20 21 20
C. 21 20 21
D. 20 21 21

(b) Which of the following statements will determine if a person?s age is between 18 (inclusive) and 60 (inclusive)? (1)

A. `if ((age > 18) && (age < 60))`
B. `if ((age > 18) || (age < 60))`
C. `if ((age >= 18) && (age <= 60))`
D. `if ((age >= 18) || (age <= 60))`

(c) Given the following C++ code: (1)

```
#include <iostream>
using namespace std;
int main() {
    int module = 1;
    switch (module) {
        case 1: cout<<"COS132";
        case 2: cout<<"COS151";
        case 3: cout<<"COS110";
        case 4: cout<<"COS122";
        default: cout<<"Invalid Module";
    }
    return 0;
}
```

When the program is run, what will be the output?

A. COS132
B. COS132COS151COS110COS122Invalid Module
C. COS132COS151COS110COS122
D. Compilation Error

(d) Consider the following switch-case statement: (1)

```
switch (x) {
    case 1: cout << "a";
    case 2: cout << "b";
    case 3: cout << "c";
    default: cout << "d"
                  << endl;
}
```

What will the output be if the value of x is 2?

A. a

B. bcd

~~C. abcd~~

D. d

(e) How many times will the following for loop execute? (1)

```
float x;
for ( x = 0.0; x <= 1.0; x += 0.2 ) {
    cout << x << endl;
}
```

0.0 ✓
0.2 ✓
0.4 ✓
0.6 ✓
0.8 ✓
1.0 ✓
1.2 X

6 times

A. ~~10 times~~

B. 6 times

C. 5 times

D. Infinite number of times

(f) What is the output of the following C++ program? (1)

```
#include <iostream>
using namespace std ;

int main() {
    int counter = 0 ;
    while (counter < 5) {
        cout << counter << " " ;
        counter --;
    }
    return 0 ;
}
```

A. 0 -1 -2 -3 -4

B. 0 1 2 3 4

~~C. No output~~

D. 0 -1 -2 -3 -4 -5 -6 -7 -8 -9 .... (because this is an infinite loop)

(g) Given the following C++ code: (1)

```
#include <iostream>
using namespace std;
int main() {
    int cc;
    for (cc = 4; cc >= 0; cc--)  {
        cout << cc;      4  3  2
        if (cc == 2)     /  /  break
            break;
    }
    return 0;
}
```

what is the output

A. 2

B. 42310

~~C. 43~~

D. 432

(h) Given the following C++ code snippet: (1)

```
tt = myfunc(aa,bb);
```

Which one of the following statements is true about arguments passed by values when calling the above function?

✓A. The variables aa and bb are passed to the function myfunc.

XB. The variables aa and bb will be updated when any modification is done in the function myfunc.

XC. Any modifications to the variables aa and bb from inside the function will not have any effect outside the function myfunc.

XD. The memory addresses of variables aa and bb are passed to the function myfunc.

(i) When more than one user-defined function can have the same name and have different parameters. This feature in the C++ programming language is known as ________ . (1)

A. Passing by references
B. Pointer overloading
C. Function overloading
D. Operator overloading

(j) Given the following C++ code: (1)

```
#include <iostream>
using namespace std;

int myfunc(int a, int b) {
    return (a * b);   6x3=18
}
float myfunc(float a, float b) {
    return (a / b);   6/3 → 2
}
int main() {
    int tt = 6, ww = 3;
    float n = 6.0, m = 3.0;
    cout << myfunc(tt, ww) << "\t";   → 18
    cout << myfunc(n, m);   → 2
    return 0;
}
```

What is the output

A. 6 3
B. 12 3
C. 18 2
D. 2 6

Question 2

[10 marks]

For each of the following multiple choice questions, write down the most appropriate option.

(a) Given the following declarations: (2)

```
int j;
int sum;
double sales[10][7];
```

Which of the following correctly finds the sum of the elements of the fifth row of sales?

A. `int sum = 0; for(j = 0; j < 7; j++)   sum = sum + sales[5][j];`
B. `int sum = 0; for(j = 0; j < 7; j++)   sum = sum + sales[4][j];`
C. `int sum = 0; for(j = 0; j < 10; j++)   sum = sum + sales[j][5];`
D. `int sum = 0; for(j = 0; j < 10; j++)   sum = sum + sales[j][4];`

(b) Consider the following array definitions: (2)

```
int values[ ] = {10, 20, 30, 40, 50};
float rates[ ] = {2.5, 9.0, 1.5, 0.5, 3.3};
char name[ ] = "COS132";
bool valid[ ] = {1, 1, 0, 0, 1};
```

Which of the cout statements will return array elements?

A. `cout << values << endl;`
B. `cout << rates << endl;`
C. `cout << name << endl;`
D. `cout << valid << endl;`

(c) Given the following array declaration. (2)

`int myarray[5][6];`

How many elements can the array store?

A. 11
B. 12
C. 25
D. 30

(d) Consider the following definition of an array: (2)

`int* myarray[5][6];`

What is the type of the elements stored in the array?

A. int
B. int*
C. int*[5]
D. int[6]

(e) Given the following array declaration and creation thereof. (2)

```
int** myarray;
myarray = new int*[5];
for (int i = 0; i < 5; i++)
    myarray[i] = new int[6];
```

What is the type of the elements stored in the array?

A. int
B. int*
C. int*[5]
D. int[6]

Question 3 [15 marks]

(a) Given the following declaration:

```
float fValues[5] = { 55.5, 65.5, 75.5, 85.5, 95.5 };
```

Suppose the C++ compiler uses 4 bytes to store a float value and that memory is allocated to store the array `fValues` starting at address 8000. What are the values of the following expressions?

i. `fValues` 8000 (1)
ii. `&fValues` 8000 (1)
iii. `*fValues` 55.5 (1)
iv. `fValues[1]` 65.5 (1)
v. `&fValues[1]` 65.5 (1)
vi. `fValues+2` 8008 (1)
vii. `*(fValues + 2)` 75.5 (1)

(b) Consider the following C++ program. (2)

```
#include <iostream>
using namespace std;

int main() {
    int numbers[] = {4, 2, 8, 5, 1};
    int* ptr = numbers;
    int value = *ptr; value=4
    for (int i = 1; i < 5; i++) {
        if (*(ptr + i) > value) {
            value = *(ptr + i);
        }
    }
    cout << "The output is: " << value << endl;   8

    return 0;
}
```

i			value=4
1	ptr+1=1	2>4	
2	ptr+2=2	8>4	value=8
3	ptr+3=3	5>8	
4	ptr+4=4	1>8	

What is the output of the program when it is run? "The output is: 8"

(c) The function below accepts a pointer to an array of characters (implemented as a C string in C++) and a character as its parameters. The cat sat on the mat

```
int getInt(char* ptr, char c) {
    int x = 0;
    while (*ptr != '\0') {
        if (*ptr == c)
            x++; 3
        ptr++;
    }
    return x; 3.
}
```

i. If the function is called with the arguments words and c and words represents the character array "The cat sat on the mat." and c represents the letter 'a'. What will the output of the following call be? (3)

"The cat..." has 3 "a"'s

```
cout << "\"" << words << "\" has " << getInt(words,c) << " \'"
     << c << "\'s." << endl;
```

ii. What character is ptr pointing to when the loop terminates? "\0" (1)

? iii. Assume the function prototype is changed to accept the array of characters (C string) by reference, rather than by value. That is as follows: (2)

stores address of pointer variable

```
int getInt(char*& ptr, char c);
```

How would the implementation of the function need to be adapted to make provision for this parameter change? double asterisk?

Question 4 [8 marks]

Consider the following struct definitions and declarations:

`struct nameType {`
`string first;`
`string last;`
`};`
	`struct courseType {`
	`string name;`
	`int callNum;`
	`int credits;`
	`char grade;`
	`};`
		`struct studentType {`
		`nameType name;`
		`double gpa;`
		`courseType course;`
		`};`

```
studentType    student;
studentType    classList[100];
courseType     course;
nameType       name;
```

Answer the following questions taking these declarations and definitions into account.

(a) Mark the following statements as valid or invalid.

i. `student.course.callNum = "CSC230";` ✓ (1)

ii. `cin >> student.name;` ✗ (1)

iii. `classList[0] = name;` ✗ (1)

iv. `cout << classList[10] << endl;` ✓ (1)

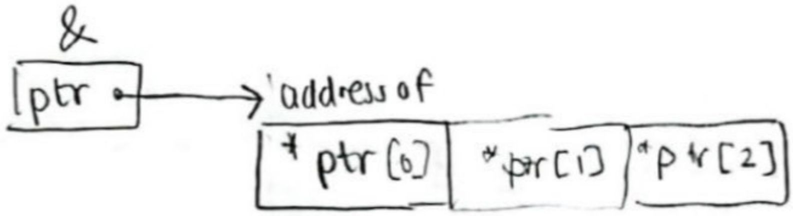
(b) Write C++ statements that do the following:

i. In the array classList, initialise each gpa to 0.0 (2)

for (int i=0; i<100; i++)
classList[i].gpa = 0.0

ii. Copy the information of the thirty-first component of the array classList into student. (2)

student = classList [31]



Long questions

Scenario

You are tasked to develop a car management system. Each car in the system is represented as a structure with the following members:

- `make`: A string representing the make of the car.
- `model`: A string representing the model of the car.
- `year`: An integer representing the year of the car.

You will be required to place the cars in a data structure and manipulate the data structure to perform the basic functions of creating, adding, querying and deleting single or multiple cars to or from the structure.

The following main program `main()` will be used to test the functions written for the car management system:

```
int main() {
    int numCars;
    cout << "Enter the number of cars: ";
    cin >> numCars;

    Car* cars = createCars(numCars);

    //Rest of the program;

    return 0;
}
```

pointer variable pointing to variable type car (struct)

Answer the questions that follow with reference to the car management system.

Question 5 [8 marks]
Defining and populating the structure

(a) Write a C++ implementation of the `Car` structure (`struct`) that includes the required members. (3)

(b) Implement a function called `createCars` for the car management system. The `createCars` function takes an integer `numCars` as input, which represents the number of cars to be created. The function should dynamically allocate an array of structures to store the information about each car. The car information is requested from the user. Below is an example of the dialogue that takes place between the system and the user. The text in **bold** is the prompt the function displays the text in *italics* is what the user types in. (5)

Enter the make of car 1: *Audi*
Enter the model of car 1: *A3*
Enter the year of car 1: *2022*

Provide a C++ implementation for the `createCars` function based on the specified requirements. The function's prototype is as follows:

```
Car* createCars(int numCars);
```

```
car* Carinfo = new car[numCars];
for (int i=0; i< numCars; i++)
{
Enter ... info
cin >> (*car[i]).make;
Enter ... info
cin >> (*car[i]).model;
Enter ... info
cin >> (*car[i]).year;
}
return Carinfo;
```

Question 6 [7 marks]
Querying the array structure

You would like to view the list of cars in the car management system on a periodic basis. To do this, you are required to write a `printCars` function. This function should produce output in the following format.

```
Cars in the database include:
Car 1 Audi S4 2022
Car 2 VW Polo 2023
```

(a) ~~Explain which of the following~~ prototypes would be valid for the `printCars` function. (2)

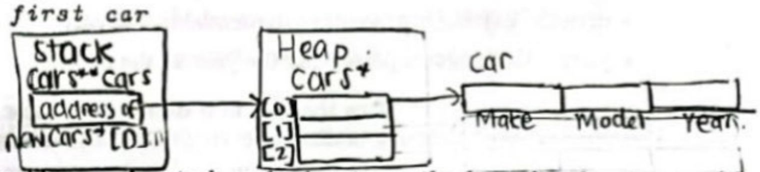
```
void printCars(const Car*, int);
void printCars(const Car[], int);
```

(b) Write the `printCars` function. (5)

Question 7 [8 marks]
A more dynamic array structure

(a) If the cars array was defined and initialised as follows:

```
int numCars;
cin >> numCars;
Cars** cars = new Cars*[numCars];
// An example of entering the first car
cars[0] = new Car;
cin >> cars[0]->Make;
cin >> cars[0]->Model;
cin >> cars[0]->Year
```



i. Indicate, by using a drawing, what resides on the stack and what is on the heap for the given initialisation of the cars structure. (4)

ii. For this definition of the cars array and the number of cars in the array, write a function that takes the array and its size as parameters and releases all the heap memory that has been allocated for the structure. The prototype of the function is given by: (4)

```
void deleteCars(Cars**, int);
```

Question 8 [11 marks]
Implementing a linked list

You decide to place the cars in a variable sized structure rather than a fixed size structure (an array). This variable sized structure is defined by linking nodes together into a list. Each Node is defined by a struct which includes a pointer to a Car and a pointer to the next node.

(a) Define a structure (struct) for Node (3)

(b) Write the code for the function insertCar that inserts a given car into a linked list structure. The head pointer for the list structure and a pointer to the car to be inserted into the list structure are sent as parameters to the function. The prototype for the insertCar function is given by: (8)

head → on heap

```
Node* insertCar(Node*, Car*);
```

returns head

The first parameter is a pointer to the head of the linked list structure. The second parameter is an instance of a Car on the heap. The function returns the head of the list after the car has been inserted. The function inserts a car into the list by grouping the same make of cars together. Note, it is not necessary for the groupings to be in alphabetic order. The cars should be grouped, by make, in the order they are entered. For example, for 8 calls to the insertCar function with the following 8 cars as input:

```
Audi A3 2022
Mercedes C200 2019
BMW 3series 2018
Audi A1 2023
Mercedes A170 2005
VW Golf 2023
VW Polo 2018
BMW X3 2023
```

The output should be as follows:

```
Car 1 Audi A3 2022
Car 2 Audi A1 2023
Car 3 Mercedes C200 2019
Car 4 Mercedes A170 2005
Car 5 BMW 3series 2018
Car 6 BMW X3 2023
Car 7 VW Golf 2023
Car 8 VW Polo 2018
```

```
Node* newNode;
Node* nodePtr;
newNode = new Node;
newNode -> pointer = car
if (!head)
    head = newNode;
else {
    nodePtr = head; → int count = 0;
    while (nodePtr -> next)
    { → count++;
        Node* compare = nodePtr -> next
        while (nodePtr -> next) → for (i, i<count, i++
        {
            if (nodePtr -> car.make = compare->carm
            {
                Node* temp = nodePtr -> next
                nodePtr -> next = compare -> next;
                compare -> next = nodePtr -> next
```